

# SCS ↔ ROS2 Bridge Architecture

How the WeighApp / JobMgr migration to ROS2 is transparent to the rest of AIS

**Key principle:** SCS is publish-subscribe over shared memory. Apps never call each other directly — they only read/write named channels. When WeighApp and JobMgr move to ROS2, the bridge takes over every SCS channel registration those two apps used to own. The other 14 apps in AIS notice nothing.

## Apps with a direct channel dependency on WeighApp or JobMgr

These 3 apps stay in AIS but need the bridge to relay their traffic to/from the migrated apps.

### LpsSaTotalsApp stays in AIS

- Reads `LpsSaLoadRecordChannel` — published by JobMgr. Bridge subscribes to ROS2 topic `/job_mgr/load_record` and republishes it on SCS.
- Writes `LpsSaWeighReqstChannel::NOTIFY_TICKET_NUMBER_WRITE` — goes to WeighApp. Bridge reads this from SCS and calls a ROS2 service on WeighApp.

### WorkOrderAssistApp stays in AIS

- Reads `LpsSaLoadRecordChannel` — same as TotalsApp, bridge handles it.
- Writes `LpsSaJobMgrReqstChannel` with commands: `WRITE_TARGET_TYPE`, `WRITE_MATERIAL_ID`, `WRITE_MATERIAL_NAME`, `WRITE_MATERIAL_DENSITY`, `WRITE_TRUCK_ID`, `WRITE_TRUCK_NAME`, `WRITE_TRUCK_TARGET_WEIGHT`. All of these go to JobMgr. Bridge reads from SCS and relays to ROS2 service.

### aisXcpServer stays in AIS — XCP calibration/debug tool

- Passively reads (snoops) 6 channels for debug/calibration visibility: `LpsSaWeighTxChannel`, `LpsSaWeighReqstChannel`, `LpsSaWeighRespChannel`, `LpsSaJobMgrTxChannel`, `LpsSaJobMgrReqstChannel`, `LpsSaJobMgrRespChannel`.
- Bridge already re-publishes all of these on SCS (because the bridge is the SCS publisher for all migrated channels), so XCP sees them unchanged.

## Apps with no dependency on WeighApp or JobMgr channels

These 13 apps are completely unaffected — they continue talking SCS to their own channels.

| App                | What it uses                                              |
|--------------------|-----------------------------------------------------------|
| LpsSaListMgrApp    | Own channels: PostInput, NotifyOutput, TxOutput           |
| WidgetApp          | CycleSegTxInterface, DataLinkData, WidgetsDebugData       |
| CycleSegApp        | Own CycleSeg channels                                     |
| DatalinkMonApp     | DataLink monitoring channels                              |
| CANTestApp         | CAN test channels                                         |
| AprilTagReader     | Camera/vision channels                                    |
| AppLogCapture      | Log capture channels                                      |
| ServiceDashboard   | Dashboard channels                                        |
| hornOnStoreTest    | Output channels only                                      |
| LpsSaCalMgrApp     | Cal channels (NVM), no direct WeighTx/JobMgrTx dependency |
| DisplayApp / UIApp | Display state channels                                    |
| SwitchInputScs     | Switch/button input channels                              |
| DataLinkData app   | PID channels                                              |

## What the bridge actually owns on SCS

The bridge is a single AIS task that holds **all** of these SCS registrations.

## Publishers — bridge → SCS

Original publishers were WeighApp / JobMgr

- `LpsSaWeighTxChannelOutput` — forwards from ROS2 `/weigh_app/tx`
- `LpsSaJobMgrTxChannelOutput` — forwards from ROS2 `/job_mgr/tx`
- `LpsSaWeighRespChannelOutput` — forwards WeighApp's ROS2 service responses
- `LpsSaJobMgrRespChannelOutput` — forwards JobMgr's ROS2 service responses
- `LpsSaLoadRecordChannelOutput` — forwards from ROS2 `/job_mgr/load_record`

## Subscribers — SCS → bridge → ROS2

Original consumers were WeighApp / JobMgr

- `LpsSaWeighReqstChannelInput` — receives commands from TotalsApp, aisXcpServer, relays to WeighApp via ROS2
- `LpsSaJobMgrReqstChannelInput` — receives commands from WorkOrderAssist, aisXcpServer, relays to JobMgr via ROS2

From the perspective of all remaining AIS apps, channel names and data types are identical. They cannot tell whether the publisher is WeighApp/JobMgr itself or the bridge re-publishing from ROS2. That is why migrating only 2 of the 16 apps works cleanly.

## Transport layer: SCS vs ROS2/DDS

SCS uses POSIX shared memory (mmap/shm\_open). There is zero TCP or UDP involved here.

Process A (WeighApp / Bridge)

```
publish()
serialize with Boost binary into
shm page
```

→  
kernel shared  
mem region  
→

Process B (JobMgr / TotalsApp / XCP)

```
get()
deserialize with Boost
```

### Every `OutputInterface<T>::publish()` call does

- Boost-serializes the struct `T` into bytes
- Writes those bytes into a named shared memory segment (identified by the channel name, e.g. `"LpsSaWeighTxChannelOutput"`)
- Bumps a sequence counter so readers know new data arrived

### Every `InputInterface<T>::get()` call does

- Checks the sequence counter
- If new data, reads + Boost-deserializes from that shared memory segment

No sockets, no pipes, no kernel buffers beyond the mmap'd page itself. This is why it's fast enough for 50 Hz WeighApp cycles.

## ROS2 side: UDP via DDS

ROS2's middleware is DDS (Fast-DDS by default, or Cyclone DDS). DDS uses UDP — multicast for discovery, unicast for data transfer.



For same-machine communication, Fast-DDS has a shared memory transport optimization that bypasses UDP entirely and goes through a *different* shared memory mechanism (not the same as SCS). Whether it kicks in depends on Fast-DDS configuration — by default it negotiates the best available transport.

## The bridge: how it connects both worlds

The bridge is one AIS task (same process group as the rest of AIS). Here is its actual data path for each direction.

### Direction 1: SCS → ROS2 (e.g. WeighApp's output reaching JobMgr)

[WeighApp ROS2 node]  
`publisher.publish(WeighTxMsg)`

↓ UDP/DDS

[Bridge ROS2 subscriber callback]  
`convert WeighTxMsg → LpsSaWeighTxChannel struct`

↓ Boost serialize

[SCS shared memory: "LpsSaWeighTxChannelOutput"]

↓ mmap read

[TotalsApp / aisXcpServer] → `InputInterface::get()`

## Direction 2: ROS2 → SCS (e.g. TotalsApp sending NOTIFY\_TICKET\_NUMBER\_WRITE to WeighApp)

[TotalsApp in AIS]

weighAppRequestOutput\_->publish(request)

↓ Boost serialize

[SCS shared memory: "LpsSaWeighReqstChannelInput"]

↓ mmap read

[Bridge — polling get() each tick]

convert LpsSaWeighReqstChannel → ROS2 service request

↓ UDP/DDS

[WeighApp ROS2 service server]

handles command, returns response

↓ UDP/DDS

[Bridge — service client gets response]

convert → LpsSaWeighRespChannel struct

↓ Boost serialize

[SCS shared memory: "LpsSaWeighRespChannelOutput"]

↓ mmap read

[TotalsApp / aisXcpServer] → InputInterface::get()

## Summary table

| Segment                              | Transport           | Protocol                               | Latency                 |
|--------------------------------------|---------------------|----------------------------------------|-------------------------|
| AIS app ↔ SCS                        | POSIX shared memory | none (mmap)                            | ~microseconds           |
| Bridge ↔ SCS                         | POSIX shared memory | none (mmap)                            | ~microseconds           |
| Bridge ↔ ROS2 node                   | DDS                 | UDP (or SHM if Fast-DDS negotiates it) | ~100μs–1ms same machine |
| ROS2 node ↔ ROS2 node (same machine) | DDS                 | UDP unicast or Fast-DDS SHM            | ~100μs–1ms              |

**Bridge scheduling:** the bridge itself runs in the AIS scheduler loop (no separate thread needed), polling SCS channels each tick and calling the ROS2 executor's `spin_some()` to drain any incoming DDS messages. No

TCP, no named pipes — it's shared memory on the SCS side and UDP/DDS on the ROS2 side, with the bridge converting between the two in memory.